

PROJET D'APPRENTISSAGE AUTOMATIQUE

Predicting Student Test Scores

Membres de l'équipe :

RHOUDAL SAAD
SEK SOPHEAK VOATEI
THONG OUSAPHEA
VAITHILINGAM VITHUSON

Enseignants responsables :

M. Blaise HANCZAR
M. Jean-Christophe JANODET

Table des matières

1	Introduction	2
2	Analyse des données	3
2.1	Statistiques descriptives	3
2.2	Analyse des corrélations et des distributions	3
3	Prétraitement des données	5
3.1	Nettoyage et sélection initiale	5
3.2	Transformation des variables	5
4	Modélisation	6
4.1	Validation Croisée	6
4.2	Régression Linéaire	6
4.3	Machine à Vecteurs de Support (SVR)	7
4.4	Arbre de Décision	8
4.5	Méthodes ensemblistes	9
4.6	Boosting	10
4.7	Stacking	11
5	Analyse des résultats et sélection du modèle	12
5.1	Interprétation des résultats	12
5.2	Sélection finale pour la soumission	12

1 Introduction

Ce rapport présente notre travail réalisé dans le cadre du projet d'Apprentissage Automatique, qui s'appuie sur la compétition Kaggle intitulée *"Playground Series - Season 6, Episode 1 : Regression with an Academic Success Dataset"*.

L'objectif principal de ce projet est de prédire le score final d'un étudiant à un examen en se basant sur de multiples caractéristiques liées à son comportement, son mode de vie et son environnement académique.

Puisque la variable cible que nous cherchons à deviner est une valeur numérique continue (un score allant de 0 à 100), nous sommes face à une tâche de régression.

Dans le cadre de notre projet, nous commencerons par une analyse des données afin d'étudier le comportement des variables. Ensuite, nous décrirons les étapes de prétraitement des données. Nous présenterons par la suite les différents algorithmes de Machine Learning implémentés ainsi que notre méthodologie d'optimisation des hyper-paramètres. Enfin, nous comparerons les performances de ces modèles afin de justifier le choix de notre algorithme final.

2 Analyse des données

Une analyse de donnée a été réalisée afin de comprendre la distribution de nos variables et d'identifier d'éventuelles valeurs atypiques. Le jeu de données d'entraînement se distingue par son volume massif, comprenant très exactement 630 000 observations.

2.1 Statistiques descriptives

Le tableau 1 résume les principales caractéristiques statistiques des variables numériques continues de notre jeu de données.

Variable	Moyenne	Écart-type	Min	Médiane (50%)	Max
Âge	20.55	2.26	17.0	21.0	24.0
Heures d'étude	4.00	2.36	0.08	4.00	7.91
Assiduité (%)	71.99	17.43	40.60	72.60	99.40
Heures de sommeil	7.07	1.74	4.10	7.10	9.90
Score final (Cible)	62.51	18.92	19.60	62.60	100.0

TABLE 1 – Statistiques descriptives des variables numériques (N = 630 000)

L'analyse de ces statistiques nous permet d'apprendre que :

- La note moyenne à l'examen est de 62.5/100, avec des scores allant de 19.6 à 100. La médiane (62.6) étant presque identique à la moyenne, cela indique que notre variable cible suit une distribution relativement symétrique (comme le montre le graphe de la distribution).
- On observe une grande disparité dans le temps de travail. L'assiduité en cours varie fortement (de 40.6% pour les moins assidus à 99.4% pour les plus rigoureux). De même, le temps de révision quotidien s'étale de quelques minutes (0.08h) à près de 8 heures (7.91h).
- Les étudiants dorment en moyenne 7 heures par nuit. Les valeurs extrêmes constituent une variance intéressante que nos algorithmes pourront exploiter pour mesurer l'impact de la fatigue sur la note finale.

2.2 Analyse des corrélations et des distributions

L'analyse visuelle de cette matrice et des distributions permet de voir que :

- On observe une corrélation positive évidente entre l'assiduité (`class_attendance`), le temps de révision (`study_hours`) et la note finale à l'examen. Cela semble logique étant donné que l'assiduité et le travail personnel permettent à l'élève de réussir un examen en général.
- Certaines variables numériques comme l'âge (`age`) présentent des coefficients de corrélation très proches de zéro avec le score final.
- Aucune variable explicative n'est parfaitement corrélée avec une autre. C'est une condition primordiale validée, car une forte multicolinéarité aurait pu rendre nos futurs modèles de régression linéaire instables ou biaisés.

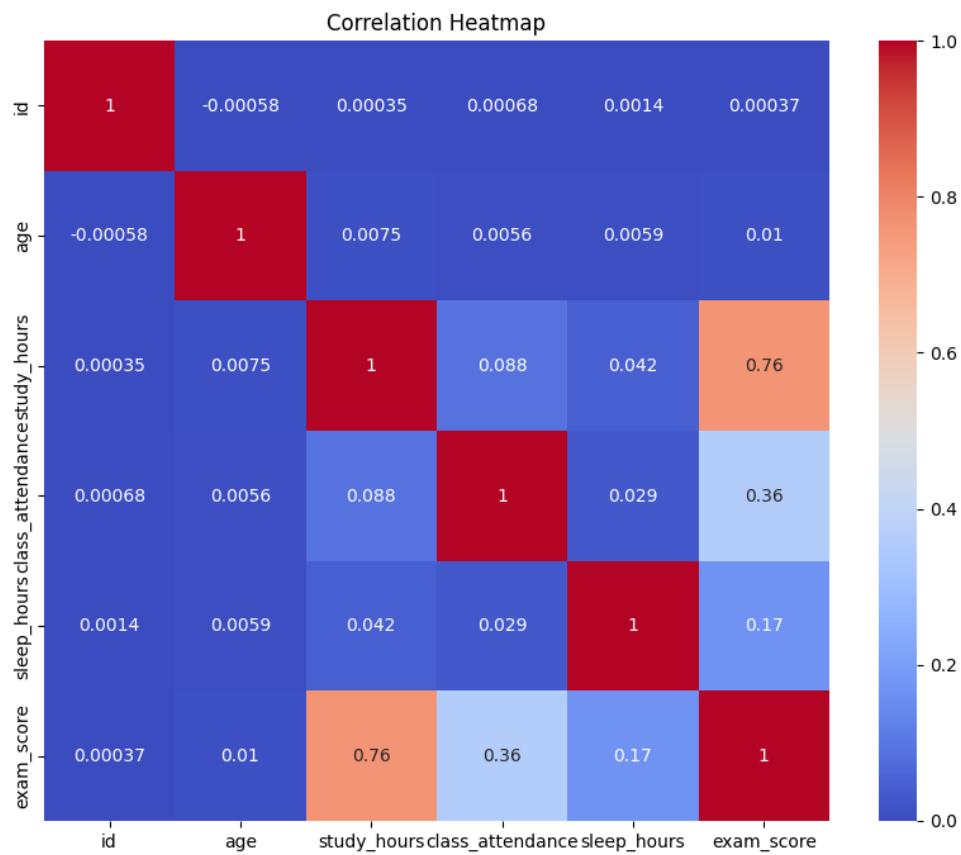


FIGURE 1 – Matrice de corrélation

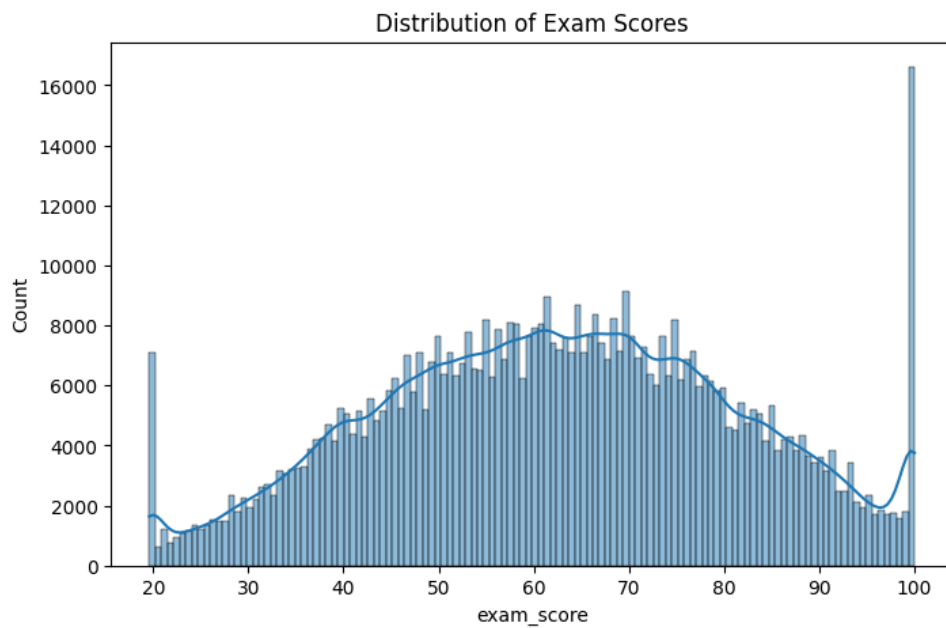


FIGURE 2 – Distribution des notes de l'examen

3 Prétraitement des données

L'analyse des données a mis en évidence le besoin de traiter certaines colonnes pour que notre base de données puisse être correctement ingérée par les modèles d'apprentissage.

3.1 Nettoyage et sélection initiale

La première étape consiste à isoler les variables utiles à la prédiction. Nous avons procédé à la suppression de la colonne `id`. En effet, cet identifiant unique attribué à chaque étudiant n'a aucune valeur prédictive.

Ensuite, nous avons séparé notre jeu de données en deux entités distinctes :

- La matrice des caractéristiques (**X**), contenant toutes les variables explicatives.
- Le vecteur cible (**y**), contenant la variable à prédire (`exam_score`).

3.2 Transformation des variables

Les algorithmes de Machine Learning ne comprennent que des nombres et sont très sensibles à l'échelle des valeurs. Étant donnée que certaines variables sont numériques, ordinales ou nominales, nous avons construit un `ColumnTransformer` comme pipeline pour transformer ces variables en variables numériques :

1. **Variables numériques :**

Les variables telles que l'âge, les heures de sommeil ou l'assiduité possèdent des échelles très différentes. Nous avons appliqué un `StandardScaler` pour centrer et réduire ces valeurs. La standardisation est obligatoire sinon la variable avec les plus grands nombres écraserait toutes les autres.

2. **Variables catégorielles ordinales :**

Certaines variables possèdent une hiérarchie logique, comme le niveau d'éducation des parents ou la difficulté de l'examen. Nous avons utilisé un `OrdinalEncoder` pour les transformer en suites de nombres (0, 1, 2) afin que l'algorithme comprenne et conserve cette notion d'ordre et de progression.

3. **Variables catégorielles nominales :**

Pour les variables n'ayant aucune relation d'ordre (comme le genre, le type d'école ou la filière d'étude), un encodage ordinal aurait créé une fausse hiérarchie. Nous avons donc opté pour un `OneHotEncoder`, qui crée une nouvelle colonne binaire (0 ou 1) pour chaque modalité.

4 Modélisation

4.1 Validation Croisée

Pour évaluer les performances de nos modèles , nous avons utilisé la méthode de la **Validation Croisée**, couplée à la fonction `GridSearchCV`.

Le principe mis en place est le suivant :

- Le jeu de données d’entraînement est divisé en K sous-ensembles de taille égale. Dans notre projet, nous avons choisi $K = 5$.
- L’algorithme s’entraîne sur $K - 1$ plis (soit 80% des données d’entraînement) et est évalué sur le pli restant.
- Cette opération est répétée 5 fois, de sorte que chaque pli serve exactement une fois de jeu de test.
- La performance finale du modèle pour une configuration donnée est la moyenne des 5 scores d’erreur obtenus.

Cette approche garantit que nos modèles généralisent bien sur de nouvelles données.

4.2 Régression Linéaire

Présentation du modèle :

L’objectif de la régression linéaire est de modéliser la note finale de l’étudiant comme une somme pondérée de ses caractéristiques . L’algorithme cherche à trouver les coefficients qui minimisent l’erreur quadratique moyenne entre ses prédictions et les vraies notes.

$$\min \left(\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \right)$$

Choix des paramètres :

La Régression Linéaire classique ne possède pas de paramètres internes pour se protéger du "bruit" causé par des variables inutiles. Pour pallier ce problème, nous l’avons intégrée dans un pipeline précédé par un filtre de sélection de variables : le `SelectKBest`.

Nous avons utilisé le test statistique `f_regression` pour évaluer la corrélation individuelle de chaque variable avec la note finale. Ensuite, nous avons demandé à notre grille de recherche (`GridSearchCV`) de tester les configurations suivantes concernant le nombre de variables à conserver :

- `k = 'all'` : Conserver et entraîner le modèle sur toutes les variables.
- `k = 10` : Ne conserver que les 10 variables les plus influentes.
- `k = 8` : Ne conserver que les 8 variables les plus influentes.

L’algorithme a ainsi pu déterminer automatiquement, grâce à la validation croisée, si le fait de retirer les variables les moins pertinentes permettait d’améliorer la précision des prédictions.

Résultat :

À l’issue de la validation croisée, le meilleur paramètre retenu par la grille de recherche est `k = 'all'`. Cela signifie que la régression linéaire a obtenu ses meilleures performances en conservant l’intégralité des caractéristiques.

4.3 Machine à Vecteurs de Support (SVR)

Présentation du modèle :

Le SVM trace une marge mathématique ϵ autour de sa droite de prédiction. La force de cet algorithme est d'ignorer totalement les étudiants dont la note réelle tombe à l'intérieur de cette droite. Le modèle considère que cette petite erreur est tolérable. L'algorithme se concentre donc exclusivement sur les cas difficiles : les étudiants dont les notes se retrouvent en dehors de la marge. Ces points que l'on appelle les vecteurs de support dictent la forme finale de l'équation.

Puisque notre objectif est de prédire une note numérique continue, nous avons utilisé la version de cet algorithme dédiée à la régression : le **SVR** (*Support Vector Regression*). Face au volume très important de nos données d'entraînement, nous avons opté pour son implémentation linéaire (**LinearSVR**), qui est optimisée pour converger beaucoup plus rapidement que la version classique.

Choix des paramètres :

Dans un jeu de données, il y a des étudiants qui ont eu un coup de chance à l'examen, ou des génies qui ont raté le coche. Lorsqu'un point se retrouve hors du tuyau, le SVR calcule une pénalité. L'hyper-paramètre C contrôle la sévérité de cette pénalité :

- Un paramètre C élevé (sévère) force le modèle à s'ajuster excessivement pour capturer tous les points hors du tuyau (risque important de surapprentissage).
- Un paramètre C plus faible (tolérant) accepte ces exceptions pour maintenir un modèle généralisable et robuste.

Résultat :

Notre grille de recherche a testé les valeurs $C \in \{0.1, 1.0, 10.0\}$. La validation croisée a déterminé que le meilleur compromis était obtenu avec $C = 1.0$.

Cette valeur modérée démontre que l'algorithme a trouvé un équilibre optimal : il sanctionne suffisamment les grosses erreurs de prédiction pour rester précis, mais maintient une marge de tolérance assez souple pour ignorer les notes atypiques et éviter le surapprentissage.

4.4 Arbre de Décision

Présentation du modèle :

Contrairement à la régression linéaire qui cherche à ajuster une équation globale, l'arbre de décision segmente l'espace des données. Il crée des règles successives (ex : si le temps de révision $> 4h$ et si l'assiduité $> 80\%$).

À chaque nœud, l'algorithme sélectionne la variable et la valeur de coupure qui réduisent le plus la variance et donc l'erreur quadratique moyenne au sein des groupes d'étudiants créés.

Choix des paramètres :

Le défaut principal d'un arbre de décision isolé est sa capacité à faire du surapprentissage. S'il est laissé libre, il grandira jusqu'à créer une règle pour presque chaque étudiant, apprenant ainsi le "bruit" par cœur. Pour éviter cela, nous avons limité sa croissance via `GridSearchCV` en optimisant ses hyper-paramètres de "taille" :

- `max_depth` : Contrôle la profondeur maximale de l'arbre, c'est-à-dire le nombre de questions successives qu'il a le droit de poser.
- `min_samples_split` : Définit le nombre minimum d'étudiants requis dans un nœud pour autoriser l'algorithme à le diviser à nouveau.

Résultat :

Les meilleurs paramètres retenus par la validation croisée sont `k = 10` pour la sélection de variables, `max_depth = 10` et `min_samples_split = 10`.

Contrairement à la Régression Linéaire qui conservait toutes les variables, l'arbre de décision a obtenu ses meilleures performances en se concentrant uniquement sur les 10 caractéristiques les plus prédictives. Un arbre est très sensible au bruit et peut facilement être perturbé par des variables peu informatives lors du choix de ses coupures.

Par ailleurs, ces contraintes strictes de profondeur (maximum 10 questions successives) et de séparation (minimum 10 étudiants pour créer une nouvelle règle) forcent le modèle à s'arrêter de grandir de manière précoce. Ainsi, chaque feuille finale regroupe suffisamment d'étudiants pour dégager une tendance générale statistiquement fiable, empêchant de fait le surapprentissage.

4.5 Méthodes ensemblistes

L'arbre de décision est sensible à une grande variance . Une légère modification des données d'entraînement peut aboutir à un arbre radicalement différent . Pour pallier cette instabilité et améliorer la robustesse de nos prédictions, nous avons utilisé des méthodes ensemblistes consistant à entraîner de multiples arbres de décision et à agréger leurs résultats .

Fôret aléatoire

Le principe de la Forêt Aléatoire repose sur le Bagging. L'algorithme crée plusieurs dizaines d'arbres indépendants. Chacun s'entraîne sur un échantillon différent de notre jeu de données (tirage avec remise) et sur un sous-ensemble aléatoire de variables.

Contrairement à nos modèles précédents, nous n'avons pas inclus de `SelectKBest` dans le pipeline du Random Forest. En effet, cet algorithme réalise sa propre sélection de caractéristiques de manière intrinsèque lors de la construction des nœuds rendant un filtrage préalable inutile.

Nous avons optimisé deux hyper-paramètres via `GridSearchCV` :

- `n_estimators` (10, 30) : Le nombre total d'arbres dans la forêt.
- `max_depth` (10, 20) : La profondeur maximale autorisée pour chaque arbre.

Résultat : Le modèle optimal a été obtenu avec `n_estimators = 30` et `max_depth = 20`. L'algorithme a privilégié le nombre maximum d'arbres proposé dans notre grille , confirmant que la multiplication des estimateurs aide à stabiliser les prédictions. De plus, grâce à la robustesse du Bagging face au surapprentissage, la forêt peut se permettre de construire des arbres profonds pour capturer des relations complexes, là où un arbre isolé aurait mémorisé le bruit statistique.

Extra Trees

Afin d'enrichir notre approche basée sur le Bagging, nous avons implémenté l'algorithme Extra Trees. Bien qu'il soit très similaire à la Forêt Aléatoire, il s'en distingue par une étape de randomisation supplémentaire : lors de la construction de chaque nœud, au lieu de chercher le seuil de coupure optimal pour chaque variable, Extra Trees choisit des seuils de manière totalement aléatoire et retient le meilleur parmi ces choix. Cette caractéristique permet de réduire encore davantage la variance du modèle tout en accélérant grandement le temps de calcul.

Résultat de l'optimisation :

La recherche des hyper-paramètres via `GridSearchCV` a permis de trouver un excellent compromis entre complexité et généralisation. Les paramètres optimaux retenus sont `n_estimators = 100` et `max_depth = 20`.

4.6 Boosting

Présentation du modèle

Contrairement au Random Forest qui construit des dizaines d'arbres indépendamment les uns des autres, le Boosting construit des arbres de manière séquentielle. Le premier arbre tente une prédiction, puis le deuxième arbre s'entraîne exclusivement à prédire et corriger les erreurs du premier, et ainsi de suite. L'algorithme se concentre donc de plus en plus sur les cas difficiles à prédire.

CatBoost

Pour éviter le surapprentissage, nous avons concentré notre recherche par validation croisée (`GridSearchCV`) sur les trois paramètres les plus influents de l'algorithme :

- Le nombre d'itérations (`iterations`) : définit le nombre total d'arbres construits séquentiellement. Une valeur trop faible empêche le modèle d'apprendre, alors qu'une valeur trop haute allonge inutilement le temps de calcul.
- La profondeur des arbres (`depth`) : contrôle la complexité des arbres symétriques. Une profondeur élevée risque de mémoriser le "bruit" statistique, alors qu'une profondeur trop faible ratera les interactions subtiles entre les variables.
- Le taux d'apprentissage (`learning_rate`) : Il contrôle l'ampleur de la mise à jour des poids à chaque itération

Résultat :

Une profondeur modérée suffit amplement à capturer les interactions entre les variables sans risquer le surapprentissage, tandis que les 200 itérations couplées à un taux d'apprentissage dynamique ont permis au modèle de converger efficacement vers une solution optimale.

LightGBM

Pour étoffer notre exploration des méthodes de Boosting, nous avons également implémenté LightGBM (*Light Gradient Boosting Machine*). Cet algorithme se distingue de CatBoost par sa stratégie de construction des arbres. Au lieu de croître niveau par niveau ou de manière symétrique, LightGBM développe ses arbres par les feuilles (*leaf-wise*). Il sélectionne toujours la feuille qui réduit le plus l'erreur globale pour la diviser, ce qui lui permet d'atteindre une très grande précision même sur des données de 630 000 données.

Résultat :

Notre optimisation a convergé vers un modèle complexe mais solidement régularisé avec `n_estimators = 400`, `learning_rate = 0.1`, `num_leaves = 63` et `subsample = 0.8`. Le nombre élevé de feuilles (63) lui permet de capturer des interactions subtiles, tandis que le sous-échantillonnage permet d'empêcher le surapprentissage.

4.7 Stacking

Principe de l'algorithme :

Le Stacking cherche à combiner les prédictions de nos meilleurs modèles . Il utilise un modèle qui apprend à pondérer les forces et faiblesses de chaque algorithme de base. Par exemple, il peut mathématiquement décider de faire confiance à CatBoost ou LightGBM pour modéliser certaines situations complexes, tout en s'appuyant sur la Régression Linéaire pour consolider la tendance générale.

Implémentation :

Nous avons intégré dans notre Stacking les meilleurs algorithmes . Pour le méta-modèle servant de juge final, nous avons opté pour une Régression Ridge . L'utilisation d'un modèle linéaire simple et régularisé pour cette étape finale est cruciale : elle empêche le Stacking de mémoriser les erreurs des modèles de base tout en ajustant automatiquement les coefficients optimaux à accorder à chaque algorithme.

5 Analyse des résultats et sélection du modèle

Pour évaluer les différents modèles , nous avons utilisé trois métriques : l'erreur absolue moyenne , le RMSE et le coefficient de détermination. On a obtenu les résultats suivants.

Modèle	MAE	RMSE	R ²
Stacking_Total	6.694	8.388	0.8022
LightGBM	6.984	8.762	0.7841
CatBoost	7.021	8.795	0.7825
LinearRegression	7.021	8.887	0.7779
SVM	7.093	8.892	0.7777
ExtraTrees	7.093	9.083	0.7680
RandomForest	7.327	9.200	0.7620
DecisionTree	7.712	9.637	0.7389

TABLE 2 – Comparaison des performances des modèles sur le jeu de validation.

5.1 Interprétation des résultats

- Le Stacking est le meilleur modèle. Ce résultat démontre que le méta-modèle a permis d'extraire le signal utile des meilleurs modèles tout en pondérant efficacement les prédictions moins précises des modèles plus simples.
- Avec un RMSE de 8.887, la Régression Linéaire simple fait partie des meilleurs modèles. Cela montre que la relation entre les variables prédictives et la note à l'examen possède une composante linéaire extrêmement forte.
- L'arbre de décision est le modèle présentant le RMSE le plus élevé à cause de sa forte variance. Bien que les méthodes de Bagging (*Random Forest* et *Extra Trees*) améliorent considérablement ce score en réduisant le surapprentissage, elles restent en retrait par rapport au Boosting. Sur ce jeu de données spécifique, la construction séquentielle des arbres s'avère bien plus efficace que la construction indépendante.

5.2 Sélection finale pour la soumission

Au vu de ces résultats, le modèle retenu pour générer nos prédictions finales sur l'ensemble de test test.csv est le Stacking. On obtient le score suivant : 8.73503

Submission and Description		Private Score ⓘ	Public Score ⓘ	Selected
 notebookd0654f7877 - v9	Complete (after deadline) · 1d ago · select	8.76398	8.73503	☐